

TOPIC NAME	PAGE	REMARK
WAP to implement ceiling and Floor Function	1	
WAP to implement fuzzy set operation	2	
WAP to implement euclidean and extended euclidean algorithm	3	
WAP to calculate binary integer add, sub, mul and division.	4	
WAP to Find the meet of two boolean matrix.	6	
WAP to test validity of argument $(p \rightarrow q) \wedge (q \rightarrow r) == (p \rightarrow r)$	7	
WAP to compute $a^n, b^n \bmod m$ by using recursion.	8	
WAP to implement Dijkstra's Algorithm.	9	
WAP to implement the binary search.	11	
WAP to implement Boolean matrix join and product	12	

I. WAP to implement ceiling and floor function.

```
#include <iostream>
```

```
int floorfunc(double number){
```

```
    int intpart = static_cast<int>(number);
```

```
    return (number < 0 || number != intpart) ? intpart - 1 : intpart;
```

```
}
```

```
int ceilfunction(double num){
```

```
    int intpart = static_cast<int>(num);
```

```
    double decimal = num num - intpart;
```

```
    if (decimal == 0) return intpart;
```

```
    else { if (decimal > 0 && decimal < 0.5) {
```

```
        return intpart;
```

```
    }
```

```
    else {
```

```
        return intpart + 1;
```

```
    }
```

```
}
```

```
}
```

```
int main(){
```

```
    double a = 5.0, b = 6.7, c = 8.2, d = -2.3; double i, j, k, l, m, n, o;
```

```
    i = floorfunc(a);
```

```
    j = floorfunc(b);
```

```
    k = floorfunc(d);
```

```
    m = ceilfunction(a);
```

```
    n = ceilfunction(b);
```

```
double p, q;
```

```
    o = ceilfunction(c);
```

```
    std::cout << "floor func of " << a << " = " << i << std::endl;
```

```
    std::cout << "floor func of " << b << " = " << j << std::endl;
```

```
    std::cout << "floor func of " << d << " = " << k << std::endl;
```

```
    std::cout << "ceil func of " << a << " = " << m << std::endl;
```

```
    std::cout << "ceil func of " << b << " = " << n << std::endl;
```

```
    std::cout << "ceil func of " << c << " = " << o << std::endl;
```

```
    return 0;
```

```
}
```

2. WAP To implement fuzzy set operation

```
#include <iostream>
#include <vector>
#include <algorithm>
using fset = std::vector<double>;
fset Funion( const fset& seta, const fset& setb ) {
    fset result;
    for ( int i=0; i < seta.size(); i++) {
        result.pushback( std::max( seta[i], setb[i] ) );
    }
    return result;
}

fset FInter( fset& seta, fset& setb ) { fset result;
    for ( int i=0; i < seta.size(); i++) {
        result.pushback( std::min( seta[i], setb[i] ) );
    }
    return result;
}

fset fcomplement( fset& seta, fset& setb ) { fset result;
    for ( double value: seta ) {
        result.pushback( 1.0 - value );
    }
    return result;
}

void Print( fset set ) { std::cout << "{ ";
    for ( double value: set ) { std::cout << value << " "; }
    std::cout << "}" << std::endl;
}

int main() {
    fset A = {0.1, 0.4, 0.8, 1.0}; B = {0.5, 0.2, 0.6, 0.9};
    fset Uset = Funion( A, B ); fset Iset = FInter( A, B );
    fset Cset = fcomplement( A );
    std::cout << "Set A:" << " "; Print( A );
    std::cout << "Set B:"; Print( B );
    std::cout << "Union set:"; Print( Uset );
    std::cout << "Intersection set:"; Print( Iset );
    std::cout << "complement of A:"; Print( Cset );
    return 0;
}
```

3. WAD to implement euclidean and extended euclidean algorithm

```
#include <iostream>
```

```
#include <tuple>
```

```
int gcd(int a, int b) {  
    while (b != 0) { int r = a % b;  
        a = b; b = r;  
    }  
    return a;  
}
```

```
std::tuple<int, int, int> EGCD(int a, int b) {  
    if (b == 0) { return std::make_tuple(a, 1, 0); }  
    else { int gcd, x1, x2;  
        std::tie(gcd, x1, x2) = EGCD(b, a % b);  
        int x = x2;  
        int y = x1 - (a / b) * x2;  
        return std::make_tuple(gcd, x, y);  
    }  
}
```

```
int main() {  
    int a, b;  
    std::cout << "Enter two number" << std::endl; std::cin >> a >> b;  
    int gcdV = gcd(a, b);  
    std::cout << "GCD OF" << a << " and " << b << " is " << gcdV << std::endl;
```

```
    int gcdE, a1, b1;  
    std::tie(gcdE, a1, b1) = EGCD(a, b);  
    std::cout << "GCD of " << a << " and " << b << " using extended "  
        << "Euclidean is" << gcdE << std::endl;  
    std::cout << "coeff of x = " << a1 << ", y = " << b1 << std::endl;  
    return 0;  
}
```

4. WAP to calculate binary interger add, sub, mul and division.

```
#include <iostream>
```

```
#include <bitset>
```

```
int bAdd(int a, int b){  
    while (b != 0){  
        int carry = a & b;  
        a = a ^ b; b = carry << 1;  
    } return a;  
}
```

```
int bSub(int a, int b){ int borrow;  
    while (b != 0){  
        borrow = (~a) & b;  
        a = a ^ b; b = borrow << 1;  
    } return a;  
}
```

```
int bMul(int a, int b){  
    int result = 0;  
    while (b > 0){  
        if (b & 1) result = bAdd(result, a);  
        a <<= 1; b >>= 1;  
    }  
    return result;  
}
```

```
int bDiv(int dividend, int divisor){  
    if (divisor == 0) {std::cout << "division by 0" << std::endl; return 1;}  
    int q = 0, r = 0;  
    for (int i = 31; i >= 0; --i){  
        r <<= 1; r |= (dividend >> i) & 1;  
        if (r >= divisor){  
            r = bSub(r, divisor);  
            q |= (1 << i);  
        }  
    }  
    return q;  
}
```

```
int main() {  
    int a, b;  
    std::cout << "enter a & b" << std::endl;  
    std::cin >> a, b;  
    std::cout << "Addition =" << bAdd(a, b) << std::endl;  
    std::cout << "Sub =" << bSub(a, b) << std::endl;  
    std::cout << "Mul =" << bMul(a, b) << std::endl;  
    std::cout << "Div =" << bDiv(a, b) << std::endl;  
    return 0;  
}
```

5 WAP to find the meet of two boolean matrix.

```
#include <iostream>
#include <vector>
using matrix = std::vector<std::vector<bool>>>;

matrix bmeet(matrix&A, matrix&B) {
    int row = A.size(), cols = A[0].size();
    matrix C(row, std::vector<bool>(cols));
    for (int i = 0; i < row; ++i) {
        for (int j = 0; j < cols; ++j) {
            C[i][j] = A[i][j] && B[i][j];
        }
    }
    return C;
}

void Print(matrix&M) {
    for (const auto& row : M) {
        for (const auto& val : row) {
            std::cout << (val ? '1' : '0') << ' ';
        }
        std::cout << std::endl;
    }
}

int main() {
    Matrix A = { {1, 0, 1}, {1, 1, 0} }, B = { {1, 1, 0}, {0, 1, 1} };
    Matrix result = bmeet(A, B);
    std::cout << "Matrix A = " << "\n";
    Print(A);
    std::cout << "Matrix B = " << "\n";
    Print(B);
    std::cout << "Result of meet = \n";
    Print(result);
    return 0;
}
```

6 WAP To test validity of argument $(p \rightarrow q) \wedge (q \rightarrow r) \Rightarrow (q \wedge r)$?
point to remember $q \rightarrow r = \neg q \vee r$

```
#include <iostream>
```

```
bool OR (bool p, bool q) { return p || q; }
```

```
bool AND (bool p, bool q) { return p & q; }
```

```
bool NOT (bool p) { return !p; }
```

```
bool IMP (bool p, bool q) { return OR (NOT (p), q); }
```

```
void Test () {
```

```
    bool values[2] = { false, true };
```

```
    std::cout << "Testing  $(p \rightarrow q) \wedge (q \rightarrow r) = q \wedge r$ : \n";
```

```
    for (bool p : values) {
```

```
        for (bool q : values) {
```

```
            for (bool r : values) {
```

```
                bool pimpliesQ = IMP (p, q);
```

```
                bool qimpliesR = IMP (q, r);
```

```
                bool left = AND (pimpliesQ, qimpliesR);
```

```
                bool right = AND (q, r);
```

```
                if (left != right) {
```

```
                    std::cout << "Fails for P=" << p << "Q=" << q << "R=" << r << "\n";
```

```
                }  
            }  
        }  
    }
```

```
    std::cout << "Pass for P=" << p << "Q=" << q << "R=" << r << "\n";
```

```
int main () {
```

```
    Test();
```

```
    return 0;
```

```
}
```

→ WAP to compute $a^n, b^n \bmod m$ by using recursion.

```
#include <iostream>

int modExp(int base, int exp, int mod) {
    if (exp == 0) return 1;
    if (exp == 1) return base % mod;
    int partial = modExp(base, exp - 1, mod);
    return (base * partial) % mod;
}

void compute(int a, int b, int n, int m, int &r1, int &r2) {
    r1 = modExp(a, n, m);
    r2 = modExp(b, n, m);
}

int main() {
    int a, b, n, m;
    int resultA, resultB;
    std::cout << "Enter a, b, n, m = ";
    std::cin std::cin >> a, b, n, m;
    if (m <= 0) { std::cerr << "modulus must be +ve" << std::endl; }
    if (n < 0) { std::cerr << "Power must be non negative\n"; }
    if (m <= 0 || n < 0) { return 1; }
    compute(a, b, n, m, resultA, resultB);
    std::cout << a << "^" << n << "% " << m << " = " << resultA
              << std::endl;
    std::cout << b << "^" << n << "% " << m << " = " << resultB
              << std::endl;
    return 0;
}
```

8. WAP to implement Dijkstra's Algorithm.

```
#include <iostream>
#include <vector>
```

```
int minDist( std::vector<int> &dist, std::vector<bool> &sptset, int V ) {
    int min = 1000000;
    int min_index = -1;
    for (int v=0; v<V; ++v) {
        if ( !sptset[v] && dist[v] <= min ) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}
```

```
void dijkstra( std::vector<std::vector<int>> &graph, int src ) {
    int V = graph.size();
    std::vector<int> dist(V, 1000000);
    std::vector<bool> sptset(V, false);
    dist[src] = 0;
    for (int count=0; count<V-1; ++count) {
        int u = minDist( dist, sptset, V );
        sptset[u] = true;
        for (int v=0; v<V; ++v) {
            if ( !sptset[v] && graph[u][v] && dist[u] != 1000000 &&
                dist[u] + graph[u][v] < dist[v] ) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    std::cout << "Vertex \t Distance from Source\n";
    for (int i=0; i<V; ++i) {
        std::cout << i << " \t " << dist[i] << std::endl;
    }
}
```

```
int main() {
```

```
int a, b;
```

```
std::vector<std::vector<int>> graph = {
```

```
{0, 10, 0, 30, 100},
```

```
{10, 0, 50, 0, 60},
```

```
{0, 50, 0, 20, 0},
```

```
{30, 0, 20, 0, 40},
```

```
{100, 60, 0, 40, 0},
```

```
};
```

```
int src = 0;
```

```
dijkstra(graph, src);
```

```
return 0;
```

```
}
```

WAP to implement the binary search

```
#include <iostream>
```

```
#include <vector>
```

```
int recursionCount=0;
```

```
int Bsearch (std::vector<int> &arr, int target, int left, int right) {  
    recursionCount++;
```

```
    if (left > right) { return -1; }
```

```
    int mid = left + (right - left) / 2;
```

```
    if (arr[mid] == target) { return mid; }
```

```
    if (arr[mid] > target) {  
        return Bsearch (arr, target, left, mid - 1);  
    }
```

```
    return Bsearch (arr, target, mid + 1, right);  
}
```

```
int main() {
```

```
    std::vector<int> arr = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
```

```
    int target=0;
```

```
    std::cout << "enter target" << std::endl;
```

```
    std::cin >> target;
```

```
    recursionCount=0;
```

```
    int result = Bsearch (arr, target, 0, arr.size() - 1);
```

```
    if (result == -1) {
```

```
        std::cout << "Element not on array" << std::endl;
```

```
    }
```

```
    else {
```

```
        std::cout << "Element " << target << " found at index  
        << result << std::endl;
```

```
    }
```

```
    return 0;
```

```
}
```

WAP to implement Boolean matrix operation join and product.

```
#include <iostream>
```

```
#include <vector>
```

```
using Matrix = std::vector<std::vector<bool>>>;
```

```
void Display(const Matrix& matrix) {
```

```
    for (auto& row: matrix) {
```

```
        for (bool element: row) {
```

```
            std::cout << element << " ";
```

```
        }
        std::cout << std::endl;
```

```
    }
```

```
Matrix Join(Matrix& A, Matrix& B) {
```

```
    int rows = A.size(), cols = A[0].size();
```

```
    Matrix Result = (rows, std::vector<bool>(cols, false));
```

```
    for (int i = 0; i < rows; ++i) {
```

```
        for (int j = 0; j < cols; ++j) {
```

```
            result[i][j] = A[i][j] || B[i][j];
```

```
        }
    }
    return Result;
```

```
}
```

```
Matrix Product(Matrix& A, Matrix& B) {
```

```
    int row = A.size(), cols = B[0].size();
```

```
    int common = A[0].size();
```

```
    Matrix result(row, std::vector<bool>(cols, false));
```

```
    for (int i = 0; i < row; ++i) {
```

```
        for (int j = 0; j < cols; ++j) {
```

```
            for (int k = 0; k < common; ++k) {
```

```
                result[i][j] = result[i][j] || (A[i][k] && B[k][j]);
```

```
            }
        }
    }
    return result;
```

```
}
```

```

int main() {
    Matrix A = { {1,0,1}, {0,1,0}, {1,1,0} };
    Matrix B = { {0,1,1}, {1,0,1}, {0,1,1} };
    std::cout << "Matrix A = " << std::endl;
    Display(A);
    std::cout << "Matrix B = " << std::endl;
    Display(B);
    std::cout << "Boolean Join = " << std::endl;
    Display(Join(A,B));
    std::cout << "Boolean Product " << std::endl;
    Display(Product(A,B));
    return 0;
}

```